DEBUGGING TOOLS IN C PROGRAMMING

Lab Report

Basic Concepts

Error: A mistake made by the programmer in source code. Bug: A flaw that causes incorrect, unexpected, or unintended program behavior. Debugging: The systematic process of finding, analyzing, and removing bugs or defects in a program.

Classification of Errors

- Syntax errors - violations of C grammar, such as a missing semicolon.
- Compilation / type errors - incompatible types or missing declarations.
- Linker errors - object files are produced, but required definitions cannot be linked.
- Runtime errors - failures during execution, such as division by zero or invalid memory access.
- Logical errors - the program runs but produces an incorrect result.

GDB Setup and Common Commands

```
sudo apt update
sudo apt install gdb
cc -g -Wall -Wextra program.c -o program
gdb ./program
```

Useful GDB commands: break (b), run (r), next (n), step (s), print (p), display, continue (c), list (l), info locals, backtrace (bt), info breakpoints, delete, disable, enable, and quit (q).

Case 1: Syntax & Compilation Error

The compiler reports invalid C syntax and identifies the approximate location of the problem.

Code (hand typed in the report)

```
#include <stdio.h>
int main()
{
    int a = 10
    printf("%d\n", a);
    return 0;
}
```

Compilation / Debugging Steps

```
cc sample1.c
# Observe the compiler diagnostic:
# error: expected ',' or ';' before 'printf'
```

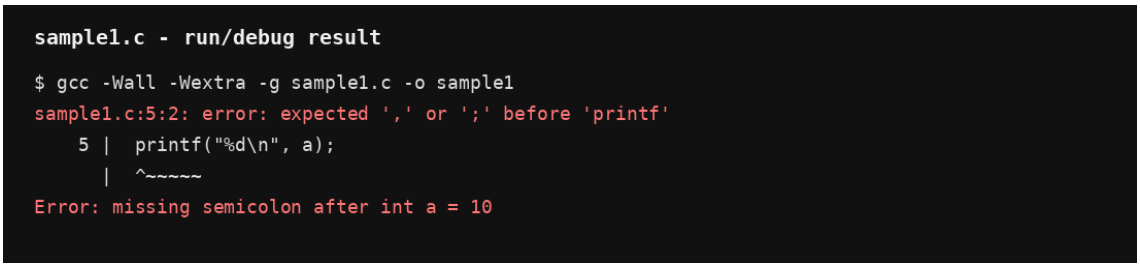


Figure: sample1.c debugging / execution output

Explanation

The declaration int a = 10 is missing a semicolon. The compiler reports the filename, line/column and an error description. Add the missing semicolon and compile again.

Corrected Code

```
#include <stdio.h>
int main()
{
    int a = 10;
    printf("%d\n", a);
    return 0;
}
```

Expected Result After Correction

```
10
```

Case 2: Compiler Warnings (-Wall, -Wextra)

Warnings identify suspicious code that is legal C but may indicate a bug.

Code (hand typed in the report)

```
#include <stdio.h>
int main()
{
    int a;
    printf("Hello\n");
    return 0;
}
```

Compilation / Debugging Steps

```
cc sample2.c
./a.out
cc -Wall -Wextra sample2.c
# warning: unused variable 'a' [-Wunused-variable]
```

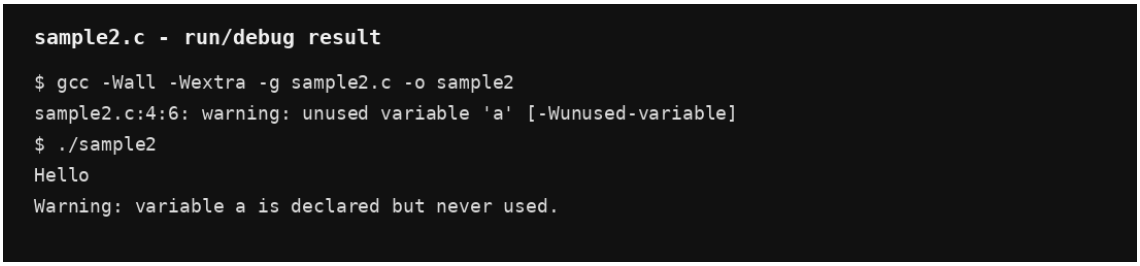


Figure: sample2.c debugging / execution output

Explanation

Normal compilation may not show minor warnings. -Wall enables a broad set of useful warnings and -Wextra enables additional warnings. Here, variable a is declared but never used.

Corrected Code

```
#include <stdio.h>
int main()
{
    printf("Hello\n");
    return 0;
}
```

Expected Result After Correction

```
Hello
```

Case 3: printf-Based Debugging (Logical Bug)

printf statements can trace program flow and variable values before using a debugger.

Code (hand typed in the report)

```
#include <stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: entering iteration i = %d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
```

```
    }
    printf("Final Sum = %d\n", sum);
    return 0;
}
```

Compilation / Debugging Steps

```
cc sample3.c -o sample3
./sample3
# Observe i and sum on every iteration.
```

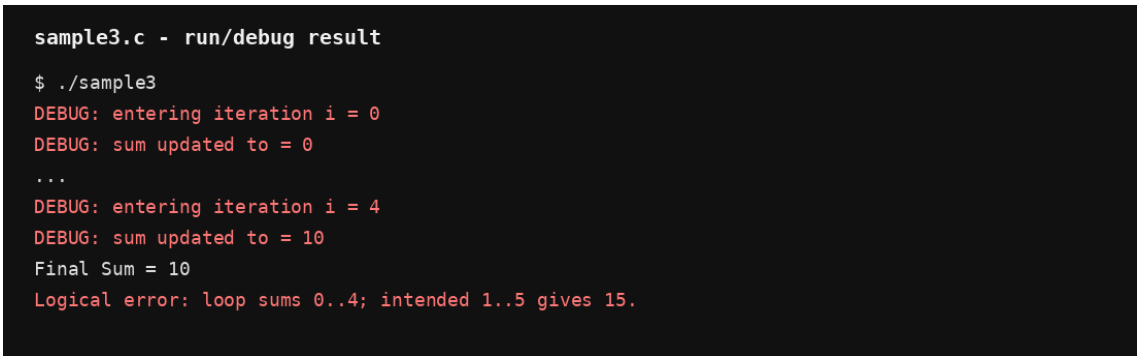


Figure: sample3.c debugging / execution output

Explanation

The trace shows that i takes the values 0, 1, 2, 3 and 4. This exposes the logical problem when the intended calculation is the sum from 1 to 5. Change the loop to start at 1 and include 5.

Corrected Code

```
#include <stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i = 1; i <= 5; i++)
        sum = sum + i;
    printf("Final Sum = %d\n", sum);
    return 0;
}
```

Expected Result After Correction

```
Final Sum = 15
```

Case 4: Debugging a Logical Error using GDB

A breakpoint and variable displays reveal why the loop gives 10 instead of the expected 15.

Code (hand typed in the report)

```
#include <stdio.h>
int main()
{
    int i, sum = 0;
    for(i = 1; i < 5; i++)
    {
        sum = sum + i;
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

Compilation / Debugging Steps

```
cc -g sample4.c -o sample4
gdb ./sample4
(gdb) list
(gdb) break 7
(gdb) run
(gdb) print i
(gdb) display i
(gdb) display sum
(gdb) continue
# Continue until the program exits.
```

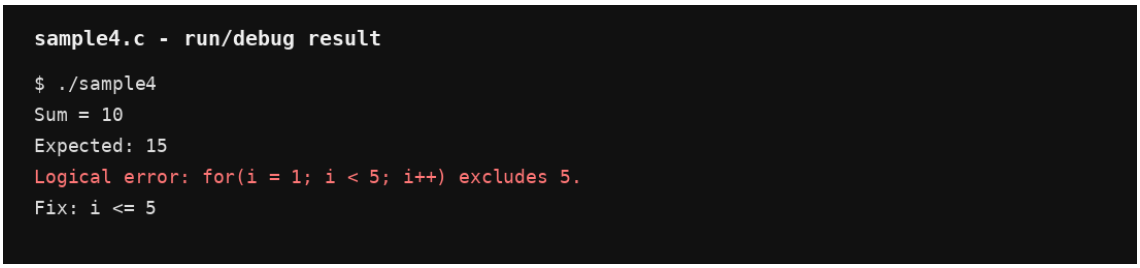


Figure: sample4.c debugging / execution output

Explanation

GDB shows that the loop body executes for i = 1 through 4. When i becomes 5, the condition i < 5 is false, so 5 is never added. Change i < 5 to i <= 5.

Corrected Code

```
#include <stdio.h>
int main()
{
```

```
    int i, sum = 0;
    for(i = 1; i <= 5; i++)
        sum = sum + i;
    printf("Sum = %d\n", sum);
    return 0;
}
```

Expected Result After Correction

```
Sum = 15
```

Case 5: Debugging Functions (next vs. step)

This case demonstrates the difference between stepping over and stepping into a function.

Code (hand typed in the report)

```
#include <stdio.h>
int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}
int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```

Compilation / Debugging Steps

```
cc -g sample5.c -o sample5
gdb ./sample5
(gdb) break main
(gdb) run
(gdb) next
(gdb) next
(gdb) print result
# Restart for step demonstration:
(gdb) run
(gdb) next
(gdb) step
(gdb) print x
(gdb) next
(gdb) print result1
```

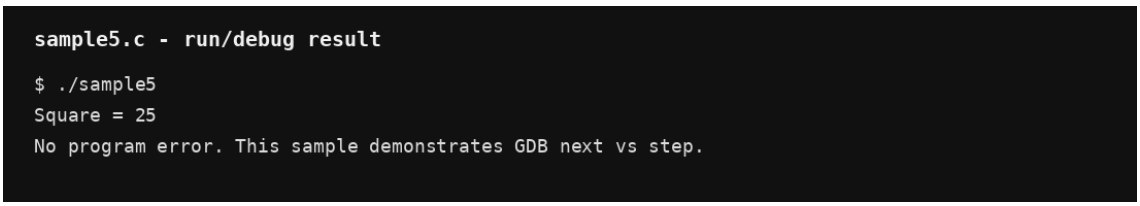


Figure: sample5.c debugging / execution output

Explanation

next executes a function call without entering its body. step enters the called function so its statements and local variables can be inspected line by line. For n = 5, square returns 25.

Expected Result After Correction

```
Square = 25
```

Case 6: Runtime Error using Backtrace (bt)

GDB backtrace identifies the chain of function calls that leads to a division-by-zero crash.

Code (hand typed in the report)

```
#include <stdio.h>
int divide(int a, int b)
{
    return a / b;
}
int calculate(int x)
{
    return divide(x, 0);
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

Compilation / Debugging Steps

```
cc -g sample6.c -o sample6
gdb ./sample6
(gdb) run
# Program receives SIGFPE
(gdb) backtrace
(gdb) frame 0
(gdb) print b
(gdb) frame 1
(gdb) print x
```

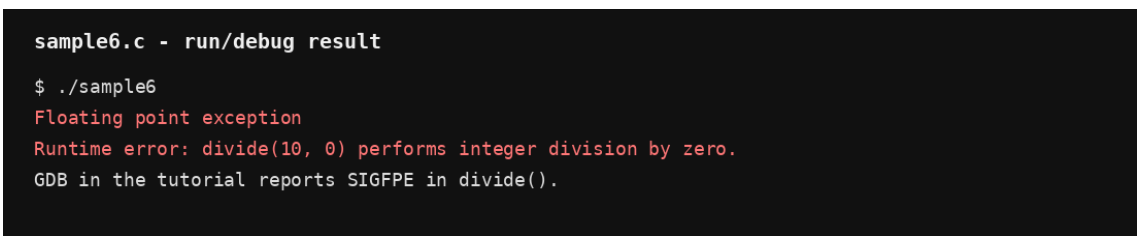


Figure: sample6.c debugging / execution output

Explanation

Frame #0 identifies divide() as the crash location and shows b = 0. Frame #1 shows calculate() passed zero as the divisor, while frame #2 is main(). The divisor must be non-zero.

Corrected Code

```
#include <stdio.h>
int divide(int a, int b)
{
    return a / b;
}
int calculate(int x)
{
    return divide(x, 2);
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

Expected Result After Correction

```
Result: 5
```

Case 7: Array Out-of-Bounds Error using GDB

C does not automatically check array bounds, so index 5 is invalid for an array of five integers.

Code (hand typed in the report)

```
#include <stdio.h>
int main()
{
    int a[5];
    int i;
    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]);
    }
    return 0;
}
```

Compilation / Debugging Steps

```
cc -g sample7.c -o sample7
gdb ./sample7
(gdb) break 8
```

```
(gdb) run
(gdb) print &a[0]
(gdb) print &a[4]
(gdb) print &a[5]
```

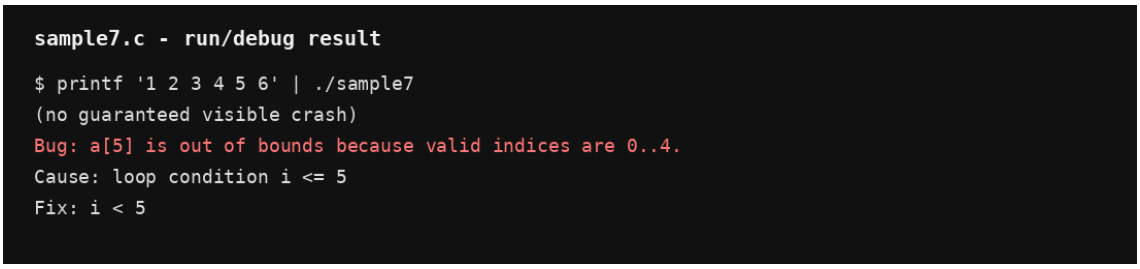


Figure: sample7.c debugging / execution output

Explanation

An array declared as a[5] has valid indices 0 through 4. The condition i <= 5 permits i = 5 and accesses memory outside the array. Change the condition to i < 5.

Corrected Code

```
#include <stdio.h>
int main()
{
    int a[5];
    int i;
    for(i = 0; i < 5; i++)
        scanf("%d", &a[i]);
    return 0;
}
```

Case 8: Segmentation Fault (SIGSEGV) using GDB

Dereferencing a NULL pointer attempts to access invalid memory and causes a segmentation fault.

Code (hand typed in the report)

```
#include <stdio.h>
int main()
{
    int *p = NULL;
    *p = 10;
    printf("%d\n", *p);
    return 0;
}
```

Compilation / Debugging Steps

```
cc -g sample8.c -o sample8
gdb ./sample8
(gdb) run
# Program receives SIGSEGV
(gdb) print p
(gdb) print *p
```

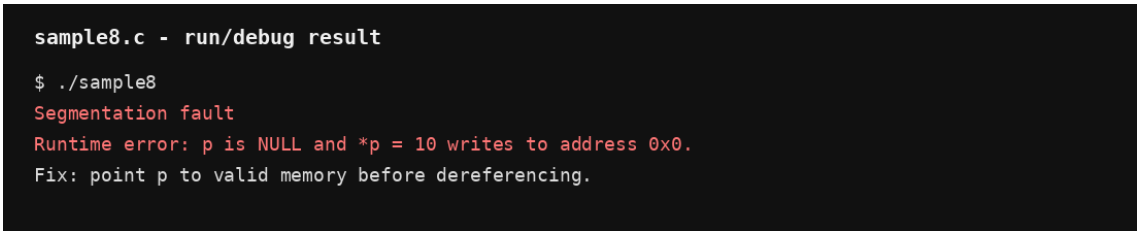


Figure: sample8.c debugging / execution output

Explanation

GDB shows p = 0x0. Dereferencing address 0x0 is invalid. Point p to valid memory before writing through it.

Corrected Code

```
#include <stdio.h>
int main()
{
    int x;
    int *p = &x;
    *p = 10;
    printf("%d\n", *p);
    return 0;
}
```

Expected Result After Correction

```
10
```

Systematic Debugging Strategy

1. Observe expected vs. actual output
2. Reproduce the issue consistently
3. Compile with flags (-Wall -Wextra -g)
4. Isolate suspicious code sections
5. Inspect state using printf or GDB breakpoints
6. Apply a single minimal fix
7. Recompile and re-test
8. Verify with edge cases and boundary inputs

SYSTEMATIC DEBUGGING STRATEGY - SOLVED

The eight steps listed in the report are explained below.

1. Observe expected vs. actual output

First identify what the program should produce and compare it with what it actually produces. The difference gives the first clue about the type and location of the bug.

2. Reproduce the issue consistently

Run the program again with the same input and conditions. A bug that can be reproduced reliably is easier to investigate because each debugging attempt can be compared under the same conditions.

3. Compile with flags (-Wall -Wextra -g)

Compile with warning and debugging options, for example: `cc -Wall -Wextra -g program.c -o program`. `-Wall` and `-Wextra` reveal suspicious code, while `-g` adds debugging information for GDB.

4. Isolate suspicious code sections

Narrow the problem to the smallest possible function, loop, statement or input. Test one section at a time so that unrelated code does not make the error harder to locate.

5. Inspect state using printf or GDB breakpoints

Use `printf` statements to display variable values and program flow, or set GDB breakpoints to pause execution. Commands such as `print`, `display`, `next` and `step` help inspect the program state.

6. Apply a single minimal fix

After locating the cause, change only what is necessary to correct it. A small, focused change makes it easier to determine whether the fix solved the original problem without introducing another bug.

7. Recompile and re-test

Compile the corrected source code again and execute it with the same test case. Confirm that the previous error is gone and that the program now gives the expected output.

8. Verify with edge cases and boundary inputs

Finally, test unusual and boundary values such as zero, negative values, minimum/maximum limits, empty input, and array boundaries. This checks that the correction works not only for the normal case but also for conditions where bugs commonly occur.

EXERCISES / HOMEWORK - SOLVED

9. Write a small C program snippet illustrating syntax, compilation/type, linker, runtime and logical errors.

(a) Syntax Error - violates the grammar of C, for example a missing semicolon.

```
#include <stdio.h>
int main() {
    int a = 10        /* missing semicolon */
    printf("%d\n", a);
    return 0;
}
```

(b) Compilation / Type Error - uses incompatible data types or an invalid declaration.

```
#include <stdio.h>
int main() {
    int x = "Hello";  /* incompatible type */
    printf("%d\n", x);
    return 0;
}
```

(c) Linker Error - a function is declared and called, but its definition is not provided.

```
#include <stdio.h>
void display(void);
int main() {
    display();        /* undefined reference at link time */
    return 0;
}
```

(d) Runtime Error - occurs while the program is executing, such as integer division by zero.

```
#include <stdio.h>
int main() {
    int a = 10, b = 0;
    printf("%d\n", a / b);
    return 0;
}
```

(e) Logical Error - the program runs but produces the wrong result.

```
#include <stdio.h>
int main() {
    int i, sum = 0;
    for (i = 1; i < 5; i++)   /* should be i <= 5 */
        sum += i;
    printf("Sum = %d\n", sum);  /* gives 10 instead of 15 */
    return 0;
}
```

10. Explain the GNU project and how GDB fits into the GNU ecosystem.

The **GNU Project** is a free-software project started to develop a complete Unix-like operating system whose software can be freely used, studied, modified and shared. GNU provides important development tools such as GCC, GDB, Bash and many command-line utilities.

GDB (GNU Debugger) is the debugging tool in the GNU development ecosystem. It helps programmers examine a program while it runs, locate errors and inspect program state. With GDB, a programmer can set breakpoints, run a program step by step, inspect variables, view the call stack using backtrace, and continue or stop execution. Programs are commonly compiled with the **-g** option so that GDB receives debugging information.

```
cc -g program.c -o program
```

```
gdb ./program
```

11. Define Shell and Kernel and list three common Linux shells such as bash, zsh and fish.

Kernel: The kernel is the core part of an operating system. It manages hardware resources such as the CPU, memory, devices and processes, and provides services that programs use to interact with the hardware.

Shell: A shell is a command interpreter that provides an interface between the user and the operating system. It accepts commands, starts programs, supports scripting and passes requests to the operating system.

Three common Linux shells: 1. **bash** - Bourne Again Shell; widely used on Linux. 2. **zsh** - Z Shell; offers powerful interactive features and customization. 3. **fish** - Friendly Interactive Shell; focuses on user-friendly interactive use, suggestions and highlighting.

12. Research and contrast Breakpoint, Watchpoint, Catchpoint and Tracepoint.

Breakpoint: Stops program execution when a specified source line, function or address is reached. It is useful for examining variables and control flow at a chosen location. Example: `break main`.

Watchpoint: Stops execution when the value of a specified variable or memory expression changes. It is useful for finding the exact statement that unexpectedly modifies data. Example: `watch x`.

Catchpoint: Stops execution when a particular event occurs rather than at a fixed source line. Depending on the target, GDB can catch events such as signals, exceptions, process creation or system calls. Example: `catch syscall`.

Tracepoint: Collects information at a selected program location for later examination, typically without stopping normal execution each time the point is reached. It is useful when repeated stopping would disturb program behavior or timing.

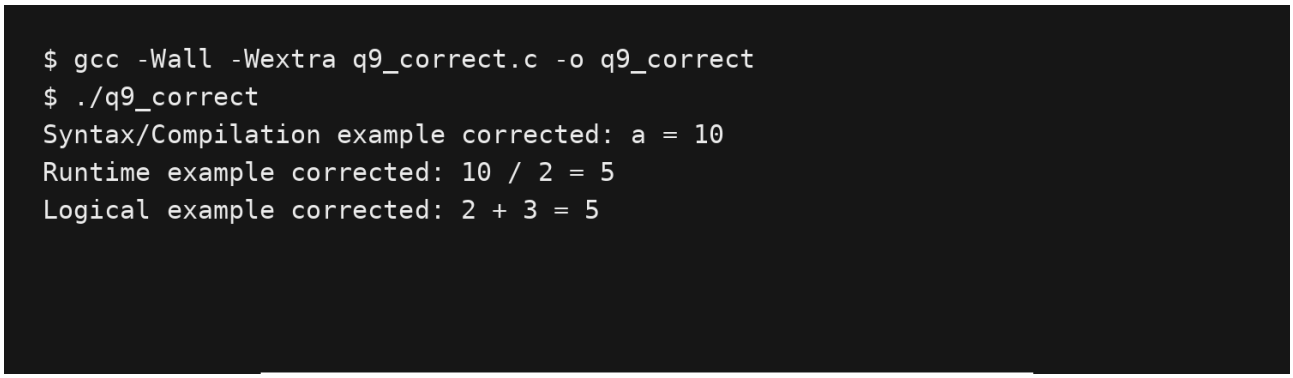
In short: a breakpoint stops at a location; a watchpoint stops when data changes; a catchpoint stops when an event occurs; and a tracepoint records selected execution information for later analysis.

Corrected Code - Compilation and Execution Result

The corrected C code was compiled with GCC using -Wall -Wextra and executed successfully.

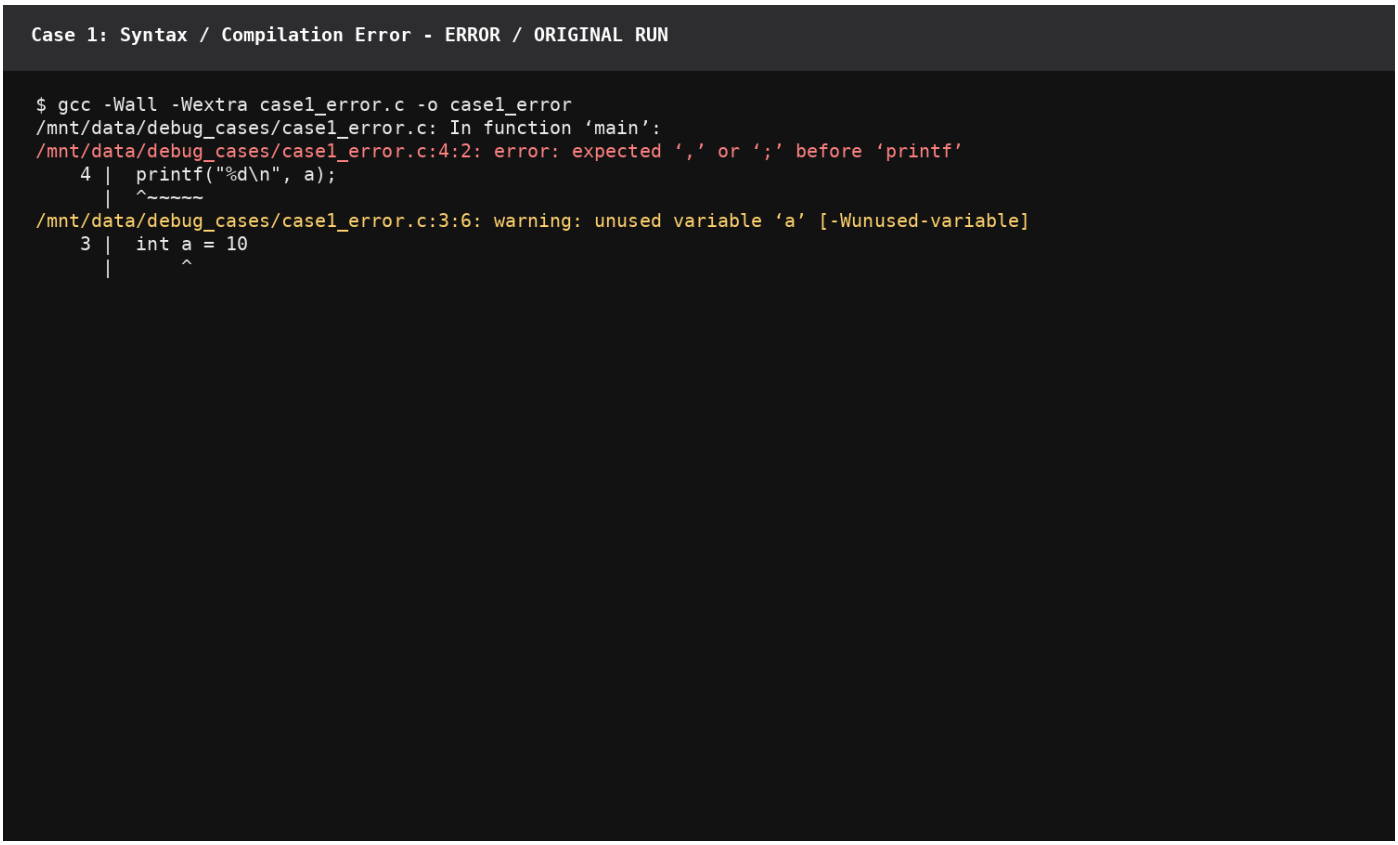
```c
#include <stdio.h>
int main(void) {
    int a = 10;
    printf("Syntax/Compilation example corrected: a = %d\n", a);
    printf("Runtime example corrected: 10 / 2 = %d\n", 10 / 2);
    printf("Logical example corrected: 2 + 3 = %d\n", 2 + 3);
    return 0;
}
```

Execution Screenshot



Case 1: Syntax / Compilation Error

Original / Error Outcome



Case 1: Syntax / Compilation Error

Corrected Outcome

Case 1: Syntax / Compilation Error - CORRECTED RUN

```
$ gcc -Wall -Wextra case1_correct.c -o case1_correct
$ ./case1_correct
10
```

Case 2: Compiler Warning

Original / Error Outcome

Case 2: Compiler Warning - ERROR / ORIGINAL RUN

```
$ gcc -Wall -Wextra case2_error.c -o case2_error
/mnt/data/debug_cases/case2_error.c: In function ‘main’:
/mnt/data/debug_cases/case2_error.c:3:6: warning: unused variable ‘a’ [-Wunused-variable]
    3 |  int a;
      |      ^
$ ./case2_error
Hello
```

Case 2: Compiler Warning

Corrected Outcome

Case 2: Compiler Warning - CORRECTED RUN

```
$ gcc -Wall -Wextra case2_correct.c -o case2_correct
$ ./case2_correct
Hello
```

Case 3: printf Logical Bug

Original / Error Outcome

Case 3: printf Logical Bug - ERROR / ORIGINAL RUN

```
$ gcc -Wall -Wextra case3_error.c -o case3_error
$ ./case3_error
DEBUG: entering iteration i = 0
DEBUG: sum updated to = 0
DEBUG: entering iteration i = 1
DEBUG: sum updated to = 1
DEBUG: entering iteration i = 2
DEBUG: sum updated to = 3
DEBUG: entering iteration i = 3
DEBUG: sum updated to = 6
DEBUG: entering iteration i = 4
DEBUG: sum updated to = 10
Final Sum = 10
```

Case 3: printf Logical Bug

Corrected Outcome

Case 3: printf Logical Bug - CORRECTED RUN

```
$ gcc -Wall -Wextra case3_correct.c -o case3_correct
$ ./case3_correct
Final Sum = 15
```

Case 4: Logical Error - Loop Condition

Original / Error Outcome

Case 4: Logical Error - Loop Condition - ERROR / ORIGINAL RUN

```
$ gcc -Wall -Wextra case4_error.c -o case4_error
$ ./case4_error
Sum = 10
```

Case 4: Logical Error - Loop Condition

Corrected Outcome

Case 4: Logical Error - Loop Condition - CORRECTED RUN

```
$ gcc -Wall -Wextra case4_correct.c -o case4_correct
$ ./case4_correct
Sum = 15
```

Case 5: Functions - next vs step

Original / Error Outcome

Case 5: Functions - next vs step - ERROR / ORIGINAL RUN

```
$ gcc -Wall -Wextra case5_error.c -o case5_error
$ ./case5_error
Square = 25

Note: Program output is correct; the debugging case demonstrates stepping behavior.
Without GDB installed in this environment, direct execution verifies the program result.
```

Case 5: Functions - next vs step

Corrected Outcome

Case 5: Functions - next vs step - CORRECTED RUN

```
$ gcc -Wall -Wextra case5_correct.c -o case5_correct
$ ./case5_correct
Square = 25
```

Case 6: Runtime Error - Division by Zero

Original / Error Outcome

Case 6: Runtime Error - Division by Zero - ERROR / ORIGINAL RUN

```
$ gcc -Wall -Wextra case6_error.c -o case6_error
$ ./case6_error
Program terminated by signal 8.
```

Case 6: Runtime Error - Division by Zero

Corrected Outcome

Case 6: Runtime Error - Division by Zero - CORRECTED RUN

```
$ gcc -Wall -Wextra case6_correct.c -o case6_correct
$ ./case6_correct
Result: 5
```

Case 7: Array Out-of-Bounds

Original / Error Outcome

Case 7: Array Out-of-Bounds - ERROR / ORIGINAL RUN

```
$ gcc -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer case7_error.c -o case7_error
$ ./case7_error
=================================================================
==1005==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7fba90000034 at pc 0x7fba9228cc05 bp
0x7fffcd6e45a0 sp 0x7fffcd6e3d60
WRITE of size 4 at 0x7fba90000034 thread T0
    #0 0x7fba9228cc04 in scanf_common
../../../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors_format.inc:342
    #1 0x7fba922c758f in __isoc99_vscanf
../../../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors.inc:1489
    #2 0x7fba922c7e6e in __isoc99_scanf
../../../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors.inc:1520
    #3 0x55a26a9cb23f in main (/mnt/data/debug_cases/case7_error+0x123f) (BuildId:
4bc59db6bc45d6c87a99777e6e874cd7edf2f7df)
    #4 0x7fba92035ca7  (/lib/x86_64-linux-gnu/libc.so.6+0x29ca7) (BuildId:
c495b62edadd6c356265942ec1282d98058a7b41)
    #5 0x7fba92035d64 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x29d64) (BuildId:
c495b62edadd6c356265942ec1282d98058a7b41)
    #6 0x55a26a9cb0c0 in _start (/mnt/data/debug_cases/case7_error+0x10c0) (BuildId:
4bc59db6bc45d6c87a99777e6e874cd7edf2f7df)

Address 0x7fba90000034 is located in stack of thread T0 at offset 52 in frame
    #0 0x55a26a9cb198 in main (/mnt/data/debug_cases/case7_error+0x1198) (BuildId:
4bc59db6bc45d6c87a99777e6e874cd7edf2f7df)

  This frame has 1 object(s):
    [32, 52) 'a' (line 2) <== Memory access at offset 52 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext
or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow
```

Case 7: Array Out-of-Bounds

Corrected Outcome

Case 7: Array Out-of-Bounds - CORRECTED RUN

```
$ gcc -Wall -Wextra case7_correct.c -o case7_correct
$ ./case7_correct
5 values stored safely.
```

Case 8: Segmentation Fault - NULL Pointer

Original / Error Outcome

Case 8: Segmentation Fault - NULL Pointer - ERROR / ORIGINAL RUN

```
$ gcc -Wall -Wextra case8_error.c -o case8_error
$ ./case8_error
Program terminated by signal 11.
```

Case 8: Segmentation Fault - NULL Pointer

Corrected Outcome